

7 - Formalizing GDScript

CS-428 project report

Mai-Linh Cordonnier 340825

Cyrielle Lozano 346639

Clément Josso 341733

Abstract

Event-driven programming languages are widely used in video games development. However, little documentation on such languages can be found. GDScript, the scripting language of Godot game engine, is an object-oriented language largely using event handling. One of the most interesting parts of its event-handling is the usage of signals between nodes to relay information. In this project, we formalize a subset of GDScript, covering variable assignment, method declaration, the `process` and ready functions, statements such as branching, but more importantly the handling of signals through awaiting and emitting. We validated our semantics through several example programs, all behaving as expected.

Keywords: PL, Big-step Semantics, Fixpoint, Godot, Event handling

1 Introduction

Video games are often reliant on scripting languages used in game engines. Such languages are frequently event-based, meaning they are driven by external events or triggers, causing specific actions to occur. GDScript uses signals[1], a delegation mechanism built into Godot that allows one game object to react to a change in another without them referencing each other. This model is suited for video game development, but nothing ensures it really behaves correctly, without introducing bugs.

This project aims to formally verify a scripting language's behavior. We initially considered Papyrus, the scripting language of Skyrim, as the game is known for its buggy quests. However, Papyrus is a proprietary language, and the documentation and information available are very limited. This made it hard for us to formalize this language, even more so considering our team had no prior knowledge of it. We therefore turned to GDScript, a well-documented open-source language. On top of that, it is a language

that we were familiar with, as we used it in former projects.

In Godot, there are two types of events: signals and `InputEvents`. `InputEvents` are hardware events that are checked before the `process` function¹ of every node, so their behavior isn't necessarily interrupting the flow of the program. However, signals are events that can be triggered by a node itself, and that can interrupt the execution of the running function until another program has been run. This behavior is what we modeled.

These signals are both callbacks and observers, which can be used in two different ways:

- We can connect signals to callable functions, such that when a signal is emitted, it instantly calls the connected function, before resuming execution.
- Nodes can wait on signals of other nodes, acting as coroutines. When an awaited signal is emitted, the waiting function resumes its execution before giving back control to the emitter.

2 Approach

`InputEvents` are hardware events that have been verified for other languages. Therefore, we turned to signals and started by choosing the subset of the grammar we needed to formalize.

2.1 Grammar

We used the official documentation of Godot and found a grammar[2] derived from the reference documentation and Godot's version 4.6. We decided to remove mostly object-oriented related features, as they weren't as interesting and relevant to our project.

¹A function run every frame, for every node.

2.1.a Top-level declarations

```
Inductive topLevelDecl :=
| classVarDecl: string → expr
  → topLevelDecl
(* method name, args,
   optional return name, body *)
| methodDecl: string → list string →
  option string → stmt → topLevelDecl
| readyDecl: stmt → topLevelDecl
| processDecl: stmt → topLevelDecl
| SequenceDecl: topLevelDecl
  → topLevelDecl → topLevelDecl
| EndDecl: topLevelDecl.
```

Listing 1: Grammar for top-level declarations

- **Global variable declaration:** for a specific node²
- **Ready function:** run only once at the beginning
- **Process function:** run every frame

We first had signal declarations that we ended up removing, even if they are part of Godot’s grammar, because of how we’re handling signals. We currently don’t have binding statements, so declaring a signal beforehand isn’t useful. We add the awaiting programs to the valuation without condition on the signal’s name, so their continuation can be run immediately when the signal is emitted.

We could reintroduce them to model Godot more closely. This would imply that when waiting or emitting a signal, we should check that the signal has been declared, and if not, stop the program.

2.1.b Statements

```
Inductive stmt :=
| varDeclStmt: string → expr → stmt
| ifStmt: expr → stmt → stmt → stmt
| whileStmt: expr → stmt → stmt
| assignmentStmt: string → expr → stmt
| awaitStmt: string → stmt
(* signal name,
   optional callback (name, node_id),
   args of signal *)
| emitSignalStmt: string →
  option (string * nat) → list expr → stmt
(* optional return variable name,
   method name, args *)
| assignCallMethodStmt: option string →
  string → list expr → stmt
| sequence: stmt → stmt → stmt
| skip: stmt.
```

Listing 2: Grammar for statements

- **Variable assignment:** change the value of an existing variable, whether local or global
- **Await:** wait for a signal to be emitted

- **Assign Call Method:** call a function and may assign a value to a given return variable
- **Emit signal:** emit a signal using its name and may execute a callback function using its arguments

2.2 Single node

2.2.a Run

Run is the function managing the top-level declarations. We chose to define it using a fixpoint with fuel instead of an inductive function, as Godot’s process functions run indefinitely. This way, when running out of fuel, we can return the last valuation of processes and use them to test our programs.

```
Inductive assignment :=
| varAss (n: nat)
| methodAss (args: list string)
  (ret: option string) (body: stmt)
| waitAss (pn: nat) (body: stmt).
```

Definition valuation := fmap string assignment.

```
Fixpoint run (fuel: nat) (v1: valuation) (d:
topLevelDecl) (v2: valuation): Prop
```

Listing 3: Initial definition of valuations and run signature

When starting to prove small programs, we encountered a problem with Qed not closing:

- Our definition used a lot of existentials, creating massive trees.

⇒ We switched the return type of run from Prop to Option of valuation, to be able to transform the existentials into pattern matching. This simplified the definition, making it easier to reason with, while reducing the size of our trees.

After this redefinition, we still had computation problems in our proofs:

- We realized that the map we took from Lab07 had an opaque definition, making it uncomputable. This meant that simplifications created a lot of branches in the trees, instead of actually computing the lookups.

⇒ We switched to a new computational map, which simplified our proofs.

We used to put the awaiting bodies in the global valuation with a special name (prepended with waiting_), and mark when a signal had been emitted in the valuation:

²We use “node” everywhere in this report for reading clarity, even though here “class” would be the correct term.

³Including in process() where multiple awaiting should be spawned.

- We couldn't await the same signal at multiple places³.
- We couldn't await the same signal twice in a row.

⇒ We extracted the signals to their own valuation, with a list of awaiting bodies, to solve both issues.

Current version:

```
Inductive assignment :=
| varAss (n: nat)
| methodAss (args: list string) (ret: option string) (body: stmt).

Inductive waiting :=
| waitAss (pn: nat) (body: stmt).

Definition valuation := fmap string assignment.

Definition waitings := fifo_map string waiting.4

Record mono_state := {
  vg: valuation;
  sig_state: waitings;
}.

Fixpoint run (fuel: nat) (ms: mono_state)
  (d: topLevelDecl) : option mono_state
```

Listing 4: Current definition of state for a single node and run signature

2.2.b Run Statement

```
Fixpoint runStmt (fuel: nat) (ms: mono_state)
  (vl: valuation) (st: stmt): option (mono_state
* valuation)
```

Listing 5: Signature of runStmt

For usual statements (if, while, variables...), our implementation is similar to the one from Lab06. The behavior of complex statements is as follows:

- Local variables shadow global ones, for declaration and assignment.
- Await does nothing outside of a sequence, but if it is followed by another statement s2, await writes s2 in the valuation as waiting for a given signal.
- AssignCallMethod calls a function by getting its body in the valuation and executing it with a local valuation only containing its arguments, possibly updating the global one. It might assign the optional return of the function to a given variable.
- Emit executes the callback with its arguments if given one, then checks for awaiting bodies. If there are any, it wakes them up in order, then resumes execution.

2.3 Two nodes

2.3.a Run Dual

At first, to run the code of two nodes simultaneously, we wrote a fixpoint runDual, which goal was to initialize both nodes (i.e. declare the variables and methods, before running the ready function once), then run both process repeatedly one after the other. However, this implementation was wrong, as it used the syntax tree as an implicit program counter, losing information when one node impacted the other.

⇒ We solved this issue by splitting the runDual fixpoint into 3 separate functions:

- initProg (called once per node), to do the initialization of a node, returning the updated state and the body of process if there is one.
- runProcessesDual, to run the body of both processes one after the other until exhaustion of the fuel.
- runDual, as a wrapper for both of these fixpoints.

Same as for run, we used to store all the shared information into global valuations (arbitrarily, we chose the valuation of the first node). The current context⁵ was written as a variable assignment, reserving the name “current”, and the state of waiting bodies was also stored in the valuation of the first node.

⇒ We applied the same change as before, creating a new record separating shared information, with the current context as a natural number, and a list of waiting bodies. This made the code clearer, more logical, easier to compute, more scalable, and removed the need for reserving names.

```
Record dual_state := {
  current : nat;
  signal_state : waitings;
  vgA : valuation;
  vgB : valuation;
}.
Fixpoint runStmtDual (fuel: nat) (ds: dual_state) (vl: valuation) (st: stmt) :
option((dual_state) * valuation)

Fixpoint initProg (fuel: nat) (ds: dual_state)
  (d: topLevelDecl) : option (dual_state * option stmt)

Fixpoint runProcessesDual (fuel: nat) (ds: dual_state) (pA pB: option stmt) : option dual_state
```

⁴Map from string to list of waiting

⁵Indicating which node is running and which valuation must be read.

```
Definition runDual (fuel: nat) (ds : dual_state)
(d: topLevelDecl * topLevelDecl) : option
(dual_state)
```

Listing 6: Current definition of state and function signatures for running two nodes

2.3.b Run Statement Dual

The working of `runStmtDual` is similar to `runStmt`, with the key difference that it checks the context in which it is run. This decides which global valuation the function will read and update when executing statements. In particular, the behavior of signal emission is significantly more complex, as it introduces context switching:

- The optional callback function bound to the emission of a signal might be defined in another node. In that case, the program needs to switch context before calling the function. It can then switch back to the original context of the node that emitted the signal before resuming execution.
- This is also the case for awaiting bodies. After calling the callback function if any, `runStmtDual` executes each body waiting for the signal in order, switching context if needed and switching back each time.

3 Results

To test our semantics, we mostly wrote small example programs to assert that they behaved the same as in Godot.

- We proved some basic examples using `runStmt`.
- We moved on to bigger programs using `run`, allowing us to use all the different statements, in particular `await` and `emit` (with and without callback) and their combinations.
- We redid the basic proofs using `runStmtDual`.
- We proved the correct behavior of a lot of programs using `runDual`. Notably:
 - Non-interacting programs run independently of each other.
 - Programs emitting signals with all possible combinations of callbacks and awaits, with and without context switching, work correctly.
- We proved that some programs that would not compile in Godot return `None` in our implementation.
- Finally, we proved the correct behavior of an actual program (Listing 7) translated from Godot to Rocq, using most of our features.

```
Class Player {
  var health: int = 11
  var dead: int = 0 #(used as a boolean)

  func takeDamage(dmg: int) -> void :
    health -= dmg
    if health == 1:
      dead = 1
}
```

```
Class Enemy {
  var strength: int = 2
  var counter: int = 0

  func hit(bonus: int) -> void :
    emit signal "hitPlayer" (callback:
      player.takeDamage(strength + bonus))

  func _process() -> void:
    if counter == 2 or counter == 4:
      hit(counter)
    counter += 1
}
```

Listing 7: Pseudo code of actual program we translated to Rocq, with the proof that `runDual` on Player and Enemy with sufficient fuel sets `player.dead` and `player.health` to 1

```
Ltac prover := vm_compute; repeat eexists; repeat
split; reflexivity.
```

Listing 8: Ltac used for all our proofs

The definition of the semantics and all the proofs can be found in the [project repository](#).

3.1 Limitations

Currently, we have some limitations compared to the standard behavior of Godot:

- We can run a maximum of 2 classes
- Connection between signals and their callback functions is done when a signal is emitted, and not by using binding statements
- Our implementation makes signal names unique for all nodes. Thus, we cannot have 2 signals with the same name in different classes. In Godot, as signals are linked to a node, different nodes can have signals with the same names.
- We cannot define a function inside another function body, which can be done in Godot (but is very rare in practice).
- We only use `nat` as primitive type.

4 Timeline

We all met up every week to work on the project together. If not specified, all team members worked on the given task.

Week 8

Planned

Papyrus feature analysis and research for existing Papyrus formalization

Actual

- Search for Papyrus documentation or existing formalization of event handling.
- Shift to GDScript when nothing conclusive was found.

Week 9-11

Planned

Definition of the semantics of our chosen features of Papyrus in Rocq

Actual

Week 9

- Decided on the subset from the grammar[2] we wanted.
- Test small programs' behavior in Godot.

Week 10

- Define inductive types for the chosen subset.
- Work on the run functions for one node.

Week 11

- Work on run functions for both one and two nodes, only missing method calls.
- Look into the formalization of interrupts.

Week 12

Planned

Encode programs and prove the correctness of our semantics

Actual

- Finish defining the semantics.
- Write example programs in Rocq, and prove the first few ones.

Week 13

Planned

Buffer week to wrap up the project and to start the final report

Actual

- Stuck with looping Qed.
- Refactor to return an option of valuations instead of Prop and transform existentials into pattern matching.
- Change our theorem definitions from using universal quantifiers to existentials.

Week 14

Planned

Presentation in class and final report

Actual

- Change Lab07 map to computational map (Clément)
- Change handling of signals, to be able to emit a signal several times (Clément)
- Creating dual_state: split current context and signal state (Clément)
- Creating the presentation slides
- Write this report

5 Related work

We found a formalization of another programming language used in video games (Lua)[3]. We also read some papers on the interrupt handling[4]. While we didn't use them, strictly speaking, they comforted us in the use of context switching.

6 Future work

We could improve our implementation to be able to run n nodes instead of 2:

- Replace global valuations A and B by a list of global valuations.
- Rewrite `runDual`, `initProg`, `runProcessesDual` and `runStmtDual` to generalize the updating of current valuation and the context switching.

Other improvements:

- Use binding statements to connect functions to signals.
- Complete our subset (pattern matching, flow, for statements).
- Expand our proof set with theorems on `runDual`'s behavior, such as:

```
∀ fuel d ms2, run fuel defaultStateMono d = Some ms2 → ∃ fuel' ds2, (runDual fuel' defaultVal (d, EndDecl)) = Some ds2 ∧ vgA ds2 = vg ms2.
```

Listing 9: Theorem that $\forall p$, `run p` is equivalent to `runDual (p, EndDecl)`

```
∀ fuel d ms21, runDual fuel defaultVal (d, EndDecl) = Some ms21 → ∃ ms22, runDual fuel defaultVal (EndDecl, d) = Some ms22 ∧ vgA ms21 = vgB ms22.
```

Listing 10: Theorem that $\forall p$, `runDual (p, EndDecl) ↔ runDual (EndDecl, p)`

7 Conclusion

We managed to define and verify our chosen subset of GDScript. At this point, we are confident in our semantics after the refactoring over the last two weeks. While the last two weeks were intense, we were able to follow the timeline we established and complete this project within the time limit.

8 AI use disclosure

We used Claude and ChatGPT to create the new map (CompMap.v) when we found in week 13 that the map from Lab07 was opaque and not computable. This was the main reason our proofs often timed out.

Bibliography

- [1] GodotDocumentation, “GDScript Documentation on Signals.” [Online]. Available: https://docs.godotengine.org/en/stable/getting_started/step_by_step/signals.html
- [2] GodotDocumentation, “GDScript Grammar.” [Online]. Available: https://docs.godotengine.org/en/stable/engine_details/file_formats/gdscript_grammar.html
- [3] M. Soldevila, B. Ziliani, B. Silvestre, D. Fridlender, and F. Mascarenhas, “Decoding Lua: formal semantics for the developer and the semanticist,” *SIGPLAN Not.*, vol. 52, no. 11, pp. 75–86, Oct. 2017, doi: [10.1145/3170472.3133848](https://doi.org/10.1145/3170472.3133848).
- [4] G. Hutton and J. Wright, “What is the Meaning of These Constant Interruptions?,” *Journal of Functional Programming*, vol. 17, no. 6, pp. 777–792, 2007, doi: [10.1017/S0956796807006363](https://doi.org/10.1017/S0956796807006363).